



MVPFY

MVPFy – Documentação completa · v0.3.0

Entrevista e plano do primeiro SaaS

Guia do usuário e referência técnica para transformar uma ideia em um plano de MVP SaaS.

Documentação completa

Edição 0.3.0

Gerado em 16/08/2026

Fonte editorial: docs/

Sumário

Documentação do MVPFy	8
Percurso	8
Ebook	8
Fonte de verdade	9
Guia completo do usuário	10
Leia online ou como ebook	10
O que o MVPFy resolve	10
Percurso pedagógico	11
Conversa contínua	11
Comece	11
A proposta do MVPFy	12
O resultado que você recebe	12
O que não faz parte	12
Por que SaaS é um recorte necessário	12
O princípio da versão 1.0	13
Uma distinção importante	13
Instalação em um projeto consumidor	14
Instalar as skills	14
Arquivos criados	14
Conferir a instalação	14
Atualização	15
Primeiro MVP: um percurso completo	16
1. Comece com uma frase	16
2. Responda uma pergunta por vez	16
3. Dê respostas ricas quando quiser	17
4. Defina a jornada principal	17
5. Defina o SaaS	18
6. Pesquise mercado e preço	18
7. Gere o MVP.md	18
8. O que você entrega às equipes	18
Entrevista adaptativa	19
Entrada inicial	19
O ciclo de cada resposta	20

Formato fixo de cada turno.....	20
Formas de resposta.....	20
Comandos de conversa.....	21
Como o fluxo evita perguntas repetidas.....	21
Quando o MVP está grande demais.....	21
O arquivo <code>MVP.md</code>	22
Estados do documento.....	22
Estrutura canônica.....	22
Fato, recomendação e hipótese.....	23
Atualização sem perda.....	23
Pesquisa de mercado, preço e custo.....	24
Com URLs de concorrentes.....	24
Sem concorrentes claros.....	24
Como o preço é montado.....	24
Tecnologia e operação recomendadas.....	26
Componentes de referência.....	26
O que precisa ser decidido no MVP.....	26
Operação assistida.....	27
Solução de problemas.....	28
A instalação não cria o estado.....	28
A pergunta seguinte repetiu uma resposta.....	28
O documento está como <code>preliminary</code>	28
Uma escolha anterior mudou.....	28
O ebook está desatualizado.....	28
Catálogo de skills do MVPFy.....	29
Ordem típica.....	29
Skills.....	29
Skill <code>mvpfy</code>	31
Quando usar.....	31
O que ela lê.....	31
O que ela faz.....	31
Regra rígida de saída.....	31
Limite sobre fontes externas ao MVPFy.....	32
Skill <code>mvpfy-problem</code>	33
Analisa.....	33

Saída	33
Exemplo	33
Não faz	33
Skill <code>mvpfy-audience</code>	34
Analisa	34
Saída	34
Exemplo	34
Não faz	34
Skill <code>mvpfy-product</code>	35
Analisa	35
Classificação	35
Exemplo	35
Não faz	35
Skill <code>mvpfy-saas</code>	36
Analisa	36
Exemplo	36
Recomendação padrão	36
Não faz	36
Skill <code>mvpfy-brand</code>	37
Entrega	37
Exemplo	37
Não faz	37
Skill <code>mvpfy-market</code>	38
Analisa	38
Saída	38
Exemplo	38
Não faz	38
Skill <code>mvpfy-pricing</code>	39
Analisa	39
Saída	39
Exemplo	39
Não faz	39
Skill <code>mvpfy-technology</code>	40
Base recomendada	40
Entrega	40

Exemplo	40
Não faz.....	40
Skill <code>mvpfy-marketing</code>	41
Entrega.....	41
Exemplo	41
Não faz.....	41
Skill <code>mvpfy-document</code>	42
Arquivos usados.....	42
Operação.....	42
Não faz.....	42
Skill <code>mvpfy-migrate</code>	43
Fluxo	43
Exemplo	43
Não faz.....	43
Guia de desenvolvimento do MVPFy	44
Leitura inicial.....	44
Para agentes.....	45
Modelo de contribuição	45
Contexto técnico.....	45
Validação rápida.....	45
Arquitetura do MVPFy	46
Fluxo de dados	46
Responsabilidades	46
Uma pergunta por turno.....	46
Fontes somente para leitura	47
Fora da arquitetura	47
Arquitetura das skills	48
Catálogo completo	48
Estrutura de uma skill.....	48
Contrato de handoff.....	49
Alterar uma skill.....	49
Não confundir com o Specsfy.....	49
Contratos.....	50
Contrato de uma pergunta	50

Estado e persistência	51
Arquivos.....	51
state.json	51
Evento de resposta	52
Gravação segura.....	52
Leitura de contexto	52
Template e MVP.md	53
IDs estáveis	53
Frontmatter	53
Regras de conteúdo.....	53
Renderização e validação	53
Migração	53
Scripts e comandos	55
Preparar o projeto.....	55
Registrar a ideia inicial	55
Registrar resposta	55
Renderizar e validar	56
Migrar	56
Testar o pacote.....	56
Mapa de arquivos do MVPFy	57
Raiz	57
Skills	57
Documentação.....	60
Pipeline do ebook	60
Governança e testes.....	61
Arquivos que o consumidor recebe	61
Versionamento e releases	62
Arquivos envolvidos.....	62
Escolha da versão	62
Como publicar uma melhoria	62
O que acontece no GitHub	63
Recusas esperadas.....	63
Diferença entre framework e ebook.....	64
Documentação e ebook	65
Ordem das páginas	65

Build	65
Quando incrementar a versão	65
Visual	65
Links	65
Testes	66
Contribuição	67
Antes de editar	67
Ao mudar comportamento	67
Ao mudar uma skill	67
Limites de escrita	67
Validação	67
Contexto técnico do MVPFy	68
Contexto do produto	69
Contexto da entrevista	70
Contexto do documento	71
Contexto de tecnologia	72
Contexto de testes	73
Especificação do MVPFy	74
Finalidade	74
Entrada e fontes	74
Contrato rígido da entrevista	74
Ciclo de persistência	75
Especialistas	75
Escopo do primeiro SaaS	75
Tecnologia preferencial	76
MVP .md	76
Migração	76
Aceite	76

Documentação do MVPFy



O MVPFy transforma uma ideia inicial em um plano utilizável para construir e validar o primeiro SaaS de uma empresa. A conversa acontece com perguntas simples, uma por vez. O trabalho especializado fica distribuído nas skills e o resultado é consolidado em um único `MVP.md`.

A documentação segue a separação usada pelo Specsfy: um percurso para quem usa o produto e outro para quem mantém a biblioteca.

Percursos

PRECISO FAZER...	DOCUMENTO
Entender a proposta e iniciar um planejamento	Guia do usuário
Instalar as skills em um projeto consumidor	Instalação
Conduzir a primeira entrevista completa	Primeiro MVP
Consultar uma skill específica	Catálogo de skills
Entender contratos, arquivos e estado	Guia técnico
Consultar a especificação funcional	Especificação

Ebook

O [ebook do MVPFy](#) reúne os dois percursos, a especificação e os exemplos em uma edição portátil. O PDF e o EPUB são gerados a partir das páginas listadas em [reading-order.txt](#).

Fonte de verdade

As skills em `skills/`, os scripts em cada pacote, os testes em `tests/` e o template `skills/mvpfy-document/assets/MVP.template.md` descrevem o comportamento executável. A documentação explica esse comportamento e não substitui os arquivos que o runtime lê.

Guia completo do usuário



O MVPFy ajuda você a sair de uma ideia vaga e chegar a um plano claro para o primeiro software oferecido como serviço. Você conversa normalmente com a skill `mvpfy`; ela faz uma pergunta por vez, guarda cada resposta e aciona os especialistas adequados para produto, mercado, preço, tecnologia e marketing.

O resultado é um `MVP.md` que outras pessoas conseguem usar para criar o website, desenhar a marca, construir a versão 1.0, vender o serviço e operar os primeiros clientes.

O MVPFy não cria `spec.md`, não executa o método do Specsify e não altera o repositório `specsify`. A semelhança entre os guias existe apenas na organização da documentação e no padrão visual do ebook.

Leia online ou como ebook

Este percurso integra a edição portátil **v0.3.0**:

- [PDF](#), para leitura e impressão;
- [EPUB](#), para leitores digitais;
- [manifesto](#), com a versão e os hashes do build.

O ebook inclui também a referência técnica e a especificação. A ordem de compilação está em [reading-order.txt](#).

O que o MVPFy resolve

No começo de uma empresa, a ideia costuma misturar público, problema, funcionalidades, preço e tecnologia. O MVPFy organiza essa conversa sem pedir que você já conheça termos como ICP, unit economics ou multitenancy.

O recorte é específico:

- uma empresa nova ou uma operação ainda em formação;
- o primeiro produto é um software;
- o software será oferecido como serviço;
- o trabalho atual é definir a versão 1.0, não todo o produto futuro.

Se a proposta for uma agência que cria sites sob encomenda, o MVPFy pergunta se existe também um produto SaaS repetível. Sem esse produto, o projeto está fora do recorte principal.

Percurso pedagógico

1. Leia [A proposta](#) para entender o que entra e o que fica fora.
2. Siga [Instalação](#) para preparar um projeto consumidor.
3. Execute [Primeiro MVP](#) com um exemplo completo.
4. Consulte [Entrevista](#) para pausar, continuar e corrigir.
5. Veja [MVP.md](#) para entender o documento produzido.
6. Use [Pesquisa e preço](#) e [Tecnologia e operação](#) quando essas áreas entrarem na conversa.
7. Consulte o [catálogo de skills](#) para entender cada especialista.

Conversa contínua

Você não precisa responder tudo em uma sessão. O MVPFy salva o estado no projeto consumidor. Quando você disser "continuar", ele lê o que já existe, identifica a pendência mais importante e faz somente a próxima pergunta.

Uma resposta rica também pode preencher várias áreas. Se você disser que agências pequenas perdem leads recebidos por WhatsApp, formulário e Instagram, o MVPFy pode aproveitar a frase para problema, público, canais e alternativa atual sem perguntar tudo novamente.

Comece

Para aprender fazendo, abra [Primeiro MVP](#). Para consultar uma regra específica, use o índice de [skills](#) ou o [guia de desenvolvimento](#).

A proposta do MVPFy

O MVPFy existe para uma situação específica: você está montando uma empresa do zero e precisa transformar a primeira ideia de software em um MVP SaaS que possa ser construído, vendido e aprendido com clientes reais.

O resultado que você recebe

Ao terminar a entrevista, o MVPFy gera um único arquivo chamado `MVP.md`. Ele funciona como uma visão compartilhada para produto, desenvolvimento, website, marca, marketing, vendas e operação. O arquivo não é uma transcrição da conversa. Ele separa o que você informou, o que foi recomendado, o que ainda é hipótese e o que precisa ser confirmado.

O documento cobre:

- empresa, contexto e modelo SaaS;
- problema, alternativa atual e hipótese de valor;
- público, comprador, pagador, usuários e personas;
- proposta, jornada principal e escopo da versão 1.0;
- conta, onboarding, ativação, assinatura, suporte e cancelamento;
- marca, nome, slogan e posicionamento inicial;
- concorrentes, preços observados e alternativas manuais;
- preço, custo operacional, receita recorrente e margem;
- Laravel, VPS, banco, arquivos, e-mail, IA e operação;
- website, conteúdo, captação, vendas e lançamento;
- métricas, validações, pendências e plano de execução.

O que não faz parte

O MVPFy não constrói o software, não publica o website, não cria um logo final e não promete resultado comercial. Ele prepara o plano para que outros agentes ou equipes executem essas tarefas.

Também não cria `spec.md`, não altera o Specsfy e não converte o `MVP.md` em uma especificação do Specsfy. O produto tem seu próprio artefato, seu próprio estado e seus próprios scripts.

Por que SaaS é um recorte necessário

Um software oferecido como serviço exige mais do que uma lista de funcionalidades. O plano precisa explicar quem cria a conta, quem usa, quem paga, como a pessoa chega ao primeiro valor, como o acesso é mantido e como o serviço é encerrado.

Durante a entrevista, o MVPFy pode recomendar processos manuais para validação. Por exemplo, uma pessoa pode liberar uma conta, acompanhar a implantação ou conferir um pagamento sem que a primeira versão automatize tudo. Essa escolha fica registrada para que a equipe saiba o que é operação provisória e o que é requisito do produto.

O princípio da versão 1.0

O MVPFy procura manter uma combinação simples: um público prioritário, um problema principal, uma promessa central e uma jornada que entrega essa promessa de ponta a ponta. Recursos adicionais entram em “Fora do MVP e evolução futura” quando não são necessários para validar o produto.

Uma distinção importante

O MVPFy pode sugerir Laravel, VPS, Supabase, armazenamento de objetos e um provedor de IA. Essas são recomendações iniciais baseadas no padrão da Promovaweb, não decisões irrevogáveis. O projeto final precisa registrar a razão de cada escolha e o gatilho para revisá-la.

Instalação em um projeto consumidor

O MVPFy é instalado no projeto que vai receber o planejamento. O repositório `mvpfy` guarda as skills e os scripts; o estado da entrevista fica no projeto consumidor.

Instalar as skills

No projeto consumidor, execute:

```
npx skills add promovaweb/mvpfy
```

Depois, inicie a estrutura local:

```
node skills/mvpfy/scripts/setup-project.mjs --project .
```

O comando prepara `.mvpfy/` e cria `MVP.md` a partir do template. Ele não altera código da aplicação, não cria `spec.md` e não escreve no repositório Specsify.

Arquivos criados

```
projeto/  
├─ MVP.md  
└─ .mvpfy/  
    ├─ config.yaml  
    ├─ state.json  
    ├─ answers.jsonl  
    └─ research.json
```

`MVP.md` é o arquivo que você compartilha com as equipes. `.mvpfy/` guarda o estado operacional para retomar a entrevista, registrar respostas e manter pesquisas. Consulte [Estado e persistência](#) para os campos e as regras de gravação.

Conferir a instalação

No repositório do MVPFy, a suíte pode ser executada com:

```
npm test
```

Em um projeto consumidor, carregue `$mvpfy` e peça para começar. A primeira resposta deve ser salva antes da pergunta seguinte. Se a estrutura não for criada, confira [Solução de problemas](#).

Atualização

Atualize as skills pelo mesmo mecanismo usado na instalação. Ao encontrar um `MVP.md` de uma versão anterior, a orquestradora compara o template e usa `mvpfy-migrate` antes de fazer novas perguntas. A atualização preserva as respostas existentes.

Primeiro MVP: um percurso completo

Este exemplo é didático. Ele não descreve um produto real do MVPFy. A ideia é uma ferramenta para pequenas agências acompanharem leads recebidos por WhatsApp, formulário e Instagram.

1. Comece com uma frase

Você pode iniciar assim:

```
Quero criar um sistema para pequenas agências que perdem leads dos clientes.
```

Ou pode enviar um contexto completo de uma vez:

```
Quero um SaaS para pequenas agências acompanharem leads dos clientes. A agência paga, o cliente acompanha apenas os próprios contatos e o MVP deve receber o lead, atribuí-lo a alguém e mostrar um resumo. Já imagino CRM, integração com WhatsApp e um painel. Quero usar Laravel em uma VPS e cobrar mensalidade.
```

O agente analisa esse texto inteiro antes de perguntar. CRM, WhatsApp e painel ficam como candidatos para investigação, enquanto problema, público, jornada, tecnologia e cobrança já podem preencher partes do plano. Você pode mandar mais mensagens antes de escolher `4.` `Continuar para as perguntas`.

O MVPFy não transforma essa frase diretamente em módulos. Ele começa entendendo a situação.

2. Responda uma pergunta por vez

Uma conversa possível:

MVPFy: Em qual situação o problema aparece com mais frequência?

1. A equipe recebe contatos em vários canais e perde o acompanhamento.
2. A equipe demora para responder ao primeiro contato.
3. A equipe não consegue mostrar resultados para o cliente.
4. Avançar
5. Conversar mais sobre este tema

Você: 1

MVPFy: resposta salva. Quem usaria o produto no trabalho diário?

1. A equipe da agência.
2. O cliente da agência.
3. A equipe e o cliente, com acessos diferentes.
4. Avançar
5. Conversar mais sobre este tema

O texto “resposta salva” representa o comportamento obrigatório. A pergunta seguinte só aparece depois do registro no `answers.jsonl` e da atualização do estado.

Se você escolher 5, pode explicar um caso, perguntar o significado de uma opção ou trazer uma resposta mais complexa. O MVPFy conversa sobre a mesma questão, salva cada turno e continua exibindo no máximo uma pergunta por vez.

3. Dê respostas ricas quando quiser

Você não precisa escolher apenas um número. Esta resposta também é válida:

A agência paga. Cada cliente deve acompanhar somente os próprios leads, e a equipe da agência precisa administrar tudo em uma conta.

Essa frase pode preencher comprador, pagador, titular da conta, usuários, permissões e isolamento entre clientes. O MVPFy não deve perguntar novamente quem paga.

4. Defina a jornada principal

Depois de entender o problema, a conversa reduz o produto a um fluxo. No exemplo, uma jornada possível é:

1. A agência cria uma conta.
2. A equipe cadastra um cliente.
3. O sistema recebe ou registra um lead.
4. A equipe atribui o lead a alguém.

5. O cliente acompanha o andamento.

6. A agência mostra o resultado em um resumo simples.

O MVPFy pergunta quais etapas são indispensáveis. Um aplicativo móvel, um marketplace e automações avançadas podem ficar para depois se não forem necessários para provar essa jornada.

5. Defina o SaaS

O plano precisa registrar que a agência é a conta pagadora, que cada cliente tem acesso limitado e que o primeiro valor ocorre quando um lead é registrado, atribuído e acompanhado. Também precisa dizer se o teste é gratuito, se a implantação será assistida e qual evento confirma ativação.

6. Pesquisa mercado e preço

Se você fornecer URLs de concorrentes, o especialista de mercado consulta planos, limites e preços atuais e registra fonte e data. Se não houver uma referência direta, a conversa pergunta quanto a agência gasta hoje com planilha, horas de acompanhamento e perda de leads.

O especialista de preço transforma esses dados em uma faixa inicial, não em uma promessa. O documento pode trazer um cenário enxuto, um cenário base e um cenário de crescimento.

7. Gere o MVP.md

Quando você pedir “gerar o documento”, o MVPFy consolida todas as áreas. Uma versão inicial pode ter estado `preliminary` e mostrar `Pendente` nas lacunas. Quando houver problema, público, jornada, escopo, modelo SaaS, preço, tecnologia, aquisição, métricas e validações suficientes, o documento pode passar a `ready`.

8. O que você entrega às equipes

O `MVP.md` permite que cada equipe comece do mesmo entendimento:

- produto recebe escopo e critérios de aceite;
- desenvolvimento recebe arquitetura, contas e integrações;
- website recebe público, promessa e oferta;
- marca recebe posicionamento, nomes e tom;
- marketing recebe canais, conteúdo e captação;
- vendas recebe preço, processo e objeções;
- operação recebe onboarding, suporte e métricas.

Entrevista adaptativa

A entrevista é a interface principal do MVPFy. Você fala em linguagem comum e a orquestradora escolhe a próxima lacuna com maior impacto no plano.

Entrada inicial

Antes da primeira pergunta com opções, descreva livremente a ideia do SaaS e do MVP. Conte o que o produto pode fazer, para quem ele serviria e quais módulos, recursos ou integrações você já imaginou. Essa é a única entrada livre de abertura. O MVPFy salva o texto e separa os itens citados como candidatos.

Depois disso, começa a entrevista com uma pergunta por vez e cinco opções. Um item citado não entra automaticamente na versão 1.0: ele será investigado e poderá entrar no MVP, ficar para depois ou ser descartado.

Você também pode enviar tudo em uma mensagem. Por exemplo:

```
Quero um SaaS para pequenas agências acompanharem leads de seus clientes.
Hoje elas usam planilhas e WhatsApp. A agência paga, cada cliente vê apenas
seus próprios leads e o MVP precisa cadastrar o lead, atribuí-lo a alguém e
mostrar um resumo. Quero começar com Laravel em uma VPS e cobrar mensalidade.
```

Nesse caso, o MVPFy registra problema, público, comprador, permissões, jornada, tecnologia e cobrança antes de perguntar. Ele não pergunta novamente quem paga nem quais etapas já foram descritas. A próxima pergunta trata apenas da lacuna mais importante que restar.

Se ainda quiser acrescentar informações, envie outra mensagem. Todas são acolhidas e salvas. Quando terminar, escolha 4. Continuar para as perguntas.

O encerramento da entrada inicial pode aparecer assim:

```
MVPFy: Entendi a ideia e já registrei o problema, o público, a jornada e os
itens candidatos. Você quer acrescentar algo antes de começar?
1. Acrescentar outro módulo ou recurso.
2. Explicar melhor o público ou o problema.
3. Informar tecnologia, preço ou canal de venda.
4. Continuar para as perguntas.
5. Conversar mais sobre este tema.
```

As opções 1, 2, 3 e 5 permitem continuar enviando conteúdo. A opção 4 salva o estado da entrada e libera a próxima pergunta fechada.

O ciclo de cada resposta

1. Você responde com número, texto, combinação de opções ou “não sei”.
2. O MVPFy guarda a resposta original.
3. A resposta é normalizada sem alterar seu sentido.
4. Fatos, escolhas, hipóteses e áreas cobertas são extraídos.
5. `state.json` e `MVP.md` são atualizados quando aplicável.
6. A orquestradora recalcula o que ainda falta.
7. Uma única pergunta seguinte é apresentada.

Se a gravação falhar, o fluxo não avança. A mesma pergunta volta para que você possa tentar novamente.

Formato fixo de cada turno

Cada pergunta aparece com cinco opções:

```
Qual situação descreve melhor o problema?  
1. A equipe perde o acompanhamento.  
2. A equipe demora para responder.  
3. A equipe não consegue mostrar o resultado.  
4. Avançar  
5. Conversar mais sobre este tema
```

As três primeiras opções são respostas prontas para a mesma pergunta. A opção 4 aceita o entendimento atual e leva à próxima etapa. A opção 5 abre um chat sobre o mesmo tema: você pode escrever uma explicação, uma dúvida ou um caso complexo. O MVPFy pode investigar esse caso em mais de um turno, mas continua exibindo no máximo uma pergunta por vez e salva cada resposta antes de seguir.

Nunca aparecem duas perguntas principais no mesmo turno.

Formas de resposta

Você pode responder:

- 1, 2 ou 3;
- 4 para avançar;
- 5 para conversar mais sobre a pergunta atual;
- “2, mas também atendo consultores”;
- uma explicação livre;

- “ainda não sei”;
- uma correção de resposta anterior;
- “pausar”, “continuar”, “revisar preço” ou “gerar documento”.

“Ainda não sei” também é informação. O MVPFy registra a lacuna e pode sugerir uma hipótese provisória, deixando claro que ela precisa de validação.

Comandos de conversa

PEDIDO	RESULTADO
“Começar”	Cria ou abre um projeto e faz a primeira pergunta útil.
“Continuar”	Lê o estado e retoma pela pendência prioritária.
“Pausar”	Salva o ponto atual sem apagar respostas.
“Revisar preço”	Trabalha somente em modelo, faixa e premissas de preço.
“Mudar o público”	Registra a correção e revisa áreas dependentes.
“O que falta?”	Mostra um resumo das pendências sem abrir formulário.
“Gerar documento”	Consolida a melhor versão, mesmo que preliminar.

Como o fluxo evita perguntas repetidas

Cada resposta recebe uma lista de campos cobertos. Uma resposta sobre “a agência paga e o cliente acompanha” cobre papéis comerciais e de produto. A orquestradora usa esses campos para retirar perguntas redundantes da fila.

Ela também verifica contradições. Se antes o comprador era uma clínica e depois você disser que venderá para agências, a alteração é registrada como correção. As personas, preço, jornada e canais relacionados voltam para revisão.

Quando o MVP está grande demais

O MVPFy pede uma escolha quando encontra várias jornadas, muitos públicos ou módulos independentes. A pergunta não é “quais recursos você quer?”, mas:

Qual resultado precisa funcionar do começo ao fim na primeira versão?

1. Registrar e acompanhar o lead.
2. Gerar o relatório para o cliente.
3. Automatizar a distribuição do lead.

As outras ideias são preservadas em “Fora do MVP e evolução futura”.

O arquivo MVP .md

MVP .md é a entrega central do MVPFy. Ele deve ser compreensível sem o histórico da conversa e útil para equipes diferentes trabalharem no mesmo MVP.

Estados do documento

ESTADO	SIGNIFICADO
preliminary	Há conteúdo aproveitável, mas faltam escolhas importantes.
ready	Os campos mínimos do primeiro SaaS estão descritos e coerentes.
validated	As hipóteses principais já foram conferidas fora da entrevista.

O estado não é uma nota de qualidade. Ele mostra quanto do plano já foi confirmado.

Estrutura canônica

O template atual contém 35 áreas:

1. resumo executivo;
2. empresa e contexto;
3. modelo da empresa SaaS;
4. problema;
5. comprovação, alternativas e hipóteses;
6. público-alvo;
7. usuário, comprador e pagador;
8. personas;
9. proposta de valor e posicionamento;
10. nome, marca e slogan;
11. jornada principal;
12. conta, acesso e isolamento;
13. onboarding e primeiro valor;
14. escopo funcional 1.0;
15. módulos e funcionalidades;
16. perfis e permissões;
17. assinatura e cobrança;
18. suporte, retenção e cancelamento;
19. processos manuais;

20. fora do MVP;
21. concorrentes e alternativas;
22. modelo comercial e preço;
23. custos e economia unitária;
24. tecnologia e arquitetura;
25. infraestrutura e operação;
26. uso de IA;
27. website;
28. marketing;
29. vendas e lançamento;
30. métricas;
31. validações e dependências;
32. execução;
33. escolhas confirmadas;
34. hipóteses e pendências;
35. fontes.

Cada seção possui um comentário estável como `<!-- mvpfy:section:problem-definition -->`. O título pode mudar sem fazer o renderer perder o conteúdo.

Fato, recomendação e hipótese

O documento usa rótulos claros:

- **Confirmado:** informação declarada por você ou comprovada por uma fonte.
- **Recomendado:** sugestão feita por uma especialista com base no contexto.
- **Hipótese:** afirmação ainda não conferida com pessoas, mercado ou uso.
- **Pendente:** dado necessário que ainda não foi fornecido.

Essa separação impede que uma estimativa de preço pareça uma pesquisa concluída ou que uma sugestão de Laravel pareça uma exigência do produto.

Atualização sem perda

Quando o template evolui, `mvpfy-migrate` insere seções novas, preserva o texto existente e marca os campos que precisam de resposta. Depois da migração, a orquestradora faz apenas uma pergunta nova por vez.

Pesquisa de mercado, preço e custo

O MVPFy pesquisa o mercado quando você informa concorrentes, URLs ou pede uma referência atual. A pesquisa apoia a conversa; ela não substitui a validação com compradores.

Com URLs de concorrentes

Informe uma URL ou nome. A skill `mvpfy-market` procura, quando disponível:

- público e promessa;
- recursos relevantes para comparação;
- preço mensal, anual ou sob consulta;
- moeda, limites e condições do plano;
- diferença entre concorrente direto, indireto e alternativa manual.

O `MVP.md` registra fonte e data. Preço ausente continua como “não publicado”; o MVPFy não inventa uma faixa e não trata promoção como preço permanente.

Sem concorrentes claros

A conversa investiga a alternativa usada hoje. Esses pontos podem entrar na fila, mas aparecem um por turno, sempre no formato de três opções, `Avançar` e `Conversar mais sobre este tema`:

- tempo gasto por mês;
- ferramentas combinadas;
- erros, perdas ou atrasos;
- resultado que teria valor suficiente para pagar.

Essa informação serve como referência de valor percebido e ajuda a escolher uma unidade de cobrança simples.

Como o preço é montado

`mvpfy-pricing` considera comprador, unidade de valor, frequência de uso, benefício, esforço comercial, custo variável, suporte e estágio da validação. Pode recomendar cobrança por conta, usuário, volume, uso ou um modelo híbrido.

O plano deve mostrar premissas e três cenários quando houver dados:

CENÁRIO	USO
Enxuto	Poucos clientes, infraestrutura mínima e operação assistida.
Base	Volume inicial esperado e custos recorrentes conhecidos.

CENÁRIO	USO
Crescimento	Aumento de uso sem introduzir complexidade antes da hora.

As contas conceituais são:

```

custo mensal = fixos + variáveis + IA + comunicação + suporte + taxas
margem por cliente = receita líquida - custo variável por cliente
clientes para equilíbrio = custos fixos / margem por cliente
MRR = soma da receita recorrente mensal dos clientes ativos
ARPA = MRR / contas pagantes

```

O resultado é uma faixa com premissas. Não é uma previsão contábil nem uma garantia de receita.

Tecnologia e operação recomendadas

O padrão inicial da Promovaweb para os projetos atendidos pelo MVPFy é Laravel em uma VPS. A recomendação favorece uma aplicação simples, custo previsível e um caminho curto até a primeira validação.

Componentes de referência

NECESSIDADE	PADRÃO INICIAL
Aplicação	Laravel.
Serviço complementar	Node.js somente com justificativa concreta.
Banco	PostgreSQL, com Supabase quando fizer sentido ao projeto.
Servidor	VPS com ambientes e backups definidos.
Arquivos	Object storage compatível com S3.
Mensagens	Provedor transacional de e-mail.
IA	Provedor externo apenas quando houver função de produto clara.
Operação	Logs, backup, filas ou tarefas agendadas conforme a jornada.

O especialista pode recomendar outra opção quando o contexto exigir, mas deve explicar o motivo e o efeito no custo e na operação.

O que precisa ser decidido no MVP

O plano não precisa desenhar cada classe do sistema. Ele precisa esclarecer:

- como a conta é criada e identificada;
- como os dados de clientes ficam separados;
- quem pode acessar cada parte;
- quais integrações são indispensáveis;
- quais tarefas podem ser assíncronas;
- onde arquivos são guardados;
- como backup, logs e suporte funcionam;
- qual custo mensal aparece em cada cenário.

Para a maioria dos primeiros SaaS, o MVPFy prefere uma aplicação compartilhada com separação lógica segura entre contas. Instâncias dedicadas só entram com uma justificativa de contrato, segurança, desempenho ou posicionamento.

Operação assistida

Durante a validação, liberar conta, acompanhar onboarding ou conferir uma assinatura manualmente pode ser aceitável. O `MVP.md` registra o processo, a pessoa responsável e o momento em que a automação passa a valer a pena.

Solução de problemas

A instalação não cria o estado

Confirme que o comando foi executado na raiz do projeto consumidor:

```
node skills/mvpfy/scripts/setup-project.mjs --project .
```

Depois confira se existem `MVP.md` e `.mvpfy/state.json`. O comando deve ser executado dentro do consumidor, não na raiz do repositório `mvpfy` para alterar o próprio pacote.

A pergunta seguinte repetiu uma resposta

Confira se a resposta anterior foi salva no `answers.jsonl` e se os campos extraídos foram atualizados no `state.json`. Uma resposta com texto livre pode precisar de interpretação; ela não deve ser descartada só porque não contém um número.

O documento está como preliminary

Abra a seção “Hipóteses e pendências”. Esse estado significa que falta pelo menos um item necessário para descrever o primeiro SaaS. Peça “continuar” ou “o que falta?” para voltar à pendência prioritária.

Uma escolha anterior mudou

Diga claramente a alteração, por exemplo: “o público agora são clínicas, não agências”. O MVPFy preserva a resposta anterior como histórico, registra a nova resposta e revisa as áreas afetadas.

O ebook está desatualizado

No repositório `mvpfy`, execute:

```
npm run ebook
npm run ebook:verify
```

Se uma página estiver ausente, confira o caminho correspondente em `docs/reading-order.txt`.

Catálogo de skills do MVPFy

O usuário conversa com `$mvpfy`. A orquestradora escolhe a especialista e integra o retorno. Você não precisa chamar cada skill manualmente, mas conhecer as responsabilidades ajuda a entender por que uma pergunta apareceu.

Ordem típica

```
contexto existente
  ↓
problema → público → produto → SaaS
  ↓         ↓         ↓         ↓
mercado → preço → tecnologia → marketing
  ↓
marca → MVP.md → migração quando o template mudar
```

Essa ordem é adaptativa. Uma fonte já existente pode preencher uma área inteira ou fazer a orquestradora voltar a um domínio anterior.

Skills

SKILL	QUANDO ENTRA	ENTREGA PRINCIPAL
mvpfy	Sempre que você inicia ou retoma	Próxima pergunta e estado integrado
mvpfy-problem	O problema ainda está genérico	Declaração do problema e hipótese
mvpfy-audience	Público ou papéis estão misturados	Segmento, ICP e personas
mvpfy-product	A solução precisa virar escopo	Jornada, módulos e versão 1.0
mvpfy-saas	É preciso explicar o serviço recorrente	Conta, onboarding e ciclo do cliente
mvpfy-brand	Produto e público já têm direção	Nome, posicionamento e slogan
mvpfy-market	Há concorrentes ou URLs	Mapa competitivo e fontes
mvpfy-pricing	Valor e cobrança precisam de hipótese	Faixas, planos e cenários
mvpfy-technology	A jornada permite desenhar a base técnica	Arquitetura Laravel e custos
mvpfy-marketing	Oferta e público precisam chegar ao mercado	Aquisição, conteúdo e venda
mvpfy-document	Você pede o plano ou há dados suficientes	<code>MVP.md</code> renderizado e validado

SKILL	QUANDO ENTRA	ENTREGA PRINCIPAL
mvpfy-migrate	O template mudou	Documento atualizado sem perda

Cada página explica entradas, análise, saída, limites, exemplo e próximo handoff. O catálogo descreve o MVPFy; ele não altera a biblioteca Specsify.

Skill `mvpfy`

`mvpfy` é a orquestradora e o único ponto de conversa. Ela não substitui o conhecimento das especialistas. Ela lê o contexto existente, escolhe o domínio necessário, controla a entrevista e consolida o resultado.

Quando usar

Use `$mvpfy` para começar, continuar, pausar, corrigir uma escolha, revisar uma área ou gerar o `MVP.md`.

O que ela lê

- `.mvpfy/config.yaml`;
- `.mvpfy/state.json`;
- `.mvpfy/answers.jsonl`;
- `MVP.md`;
- template atual;
- specs, backlogs, briefs, docs, tickets e planos já existentes no projeto consumidor, sempre em modo somente leitura.

O que ela faz

1. Confere se o projeto é um primeiro SaaS.
2. Reaproveita informação já disponível.
3. Verifica se o template precisa de migração.
4. Escolhe uma lacuna prioritária.
5. Exibe exatamente uma pergunta principal.
6. Registra a resposta antes de prosseguir.
7. Recalcula áreas cobertas e conflitos.
8. Chama a especialista adequada.

Regra rígida de saída

Uma resposta pode conter confirmação, contexto curto e opções, mas somente uma pergunta que pede resposta. Ela sempre termina com três opções prontas, `4. Avançar` e `5. Conversar mais sobre este tema`. Nunca apresente um formulário, duas perguntas encadeadas ou “responda A e B”. Se duas áreas estiverem pendentes, escolha a mais importante e deixe a outra para o próximo turno.

`Avançar` encerra a etapa com o entendimento atual. `Conversar mais sobre este tema` abre texto livre sobre a mesma pergunta e não autoriza uma segunda pergunta no turno.

Limite sobre fontes externas ao MVPFy

Specs, backlogs e documentos do projeto podem ser lidos como contexto. O MVPFy não cria, altera, renomeia, remove ou migra esses arquivos. O único documento final que ele escreve é `MVP.md`, além do estado em `.mvpfy/`.

Skill mvpfy-problem

Esta especialista transforma uma descrição de solução em um problema de negócio observável. “Preciso de um aplicativo” é uma entrada; a saída precisa explicar qual situação gera perda, atraso, erro, custo ou insegurança.

Analisa

- situação e frequência;
- pessoa afetada;
- tarefa que ela tenta realizar;
- impacto atual;
- alternativa usada hoje;
- motivo da insuficiência;
- comprovações já disponíveis;
- hipótese que o MVP deve testar.

Saída

```
Para [público], que precisa [tarefa], o problema é [dificuldade], causando [impacto]. Hoje usa [alternativa], que falha porque [limitação]. O MVP deve testar [hipótese].
```

Exemplo

“Agências perdem leads” ainda precisa de frequência, consequência e alternativa. Se o contexto existente já disser que os contatos chegam por três canais e são controlados em planilhas, a especialista usa essa informação e pergunta apenas o ponto decisivo que faltar.

Não faz

Não escolhe framework, não cria módulos e não define preço. Faz handoff para `mvpfy-audience` quando o grupo afetado estiver claro.

Skill `mvpfy-audience`

Esta especialista separa mercado amplo, segmento prioritário, usuário, comprador, pagador, influenciador e beneficiário. Ela evita personas genéricas como “qualquer empresa”.

Analisa

- frequência e intensidade do problema por segmento;
- acesso ao público;
- capacidade de pagar;
- comportamento atual;
- objeções e gatilhos;
- canais de comunicação;
- diferenças entre uso e compra.

Saída

O `MVP.md` recebe um segmento prioritário e, no máximo, três personas úteis: produto, compra e marketing quando forem diferentes. Cada persona tem contexto, objetivo, dor, alternativa, objeção, gatilho e canal.

Exemplo

Na ferramenta para agências, a agência pode ser compradora e pagadora, a pessoa de atendimento pode ser usuária diária e o cliente da agência pode ser um usuário com permissão limitada. Essa distinção afeta produto, SaaS, preço e marketing.

Não faz

Não inventa biografias para preencher espaço e não cria três públicos prioritários sem uma escolha. Faz handoff para `mvpfy-product` quando houver um segmento e um resultado central.

Skill mvpfy-product

Esta especialista converte problema e público em uma primeira versão utilizável e vendável. Sua unidade de trabalho é a jornada principal, não a lista de recursos desejados.

Analisa

- promessa central;
- entrada, processamento e saída;
- jornada de ponta a ponta;
- evento de ativação;
- módulos essenciais;
- perfis e permissões;
- regras de negócio;
- integrações indispensáveis;
- itens posteriores.

Classificação

CLASSE	USO
Essencial	Sem o item, a promessa não é entregue.
Necessária	Sustenta operação, segurança ou cobrança inicial.
Posterior	Pode esperar aprendizado.
Descartada	Não atende ao problema prioritário.

Exemplo

No caso dos leads, registrar um contato, atribuí-lo e mostrar seu andamento podem compor a jornada. Aplicativo móvel e marketplace não entram apenas porque seriam interessantes.

Não faz

Não define sozinho quem paga ou como a conta é criada. Encaminha essas dependências para `mvpfy-saas` e depois recebe as restrições de volta.

Skill mvpfy - saas

Esta especialista define como o software se comporta como serviço recorrente. Ela completa a jornada funcional com a jornada comercial e operacional.

Analisa

- B2B, B2C ou profissionais independentes;
- titular da conta e usuários;
- separação dos dados entre contas;
- convite, demonstração ou teste;
- onboarding e primeiro valor;
- ativação;
- unidade de valor;
- assinatura, limites e excedentes;
- cancelamento, inadimplência e suporte;
- processos manuais aceitáveis durante a validação.

Exemplo

Uma agência pode criar a conta, adicionar seus clientes e manter a cobrança em seu próprio contrato. Os clientes acompanham os leads, mas não enxergam dados de outras contas.

Recomendação padrão

Para muitos primeiros produtos, uma aplicação compartilhada com separação lógica segura é suficiente. Instância dedicada exige justificativa de segurança, contrato, desempenho ou posicionamento.

Não faz

Não presume que todo SaaS precisa de billing totalmente automático no primeiro dia. Se a operação assistida validar a oferta, ela pode ser registrada como provisória.

Skill mvpfy-brand

Esta especialista prepara a direção da marca depois que problema, público e promessa têm base suficiente. O resultado orienta naming e comunicação, sem afirmar disponibilidade de domínio ou marca registrada sem pesquisa.

Entrega

- categoria;
- posicionamento em uma frase;
- benefício central;
- diferenciais;
- personalidade e tom;
- palavras a usar e evitar;
- critérios para o nome;
- sugestões de nome quando solicitadas;
- slogans curtos;
- direção visual conceitual.

Exemplo

Se o valor for dar visibilidade aos leads para agências pequenas, o posicionamento deve falar de acompanhamento e resultado, sem depender de uma promessa genérica de inteligência artificial.

Não faz

Não cria logo final, não verifica disponibilidade automaticamente e não deixa o nome substituir a definição do problema.

Skill mvpfy-market

Esta especialista pesquisa concorrentes e alternativas quando você fornece referências ou pede dados atuais. Ela registra fatos observados com fonte e data e separa inferência de informação publicada.

Analisa

- concorrente direto, indireto e alternativa manual;
- público e promessa;
- recursos comparáveis;
- preço, moeda, periodicidade e limites;
- lacuna percebida;
- condições promocionais ou preço sob consulta.

Saída

Uma tabela no `MVP.md` com referência, tipo, público, promessa, recursos, preço, lacuna, link e data da consulta.

Exemplo

Se um concorrente mostra “US\$ 49 por mês para cinco usuários”, isso entra como preço observado com sua moeda e limite. Se a página só diz “fale com vendas”, o documento registra preço não publicado.

Não faz

Não inventa preço, não trata uma página antiga como atual e não inclui um recurso no MVP apenas porque um concorrente o possui.

Skill mvpfy-pricing

Esta especialista transforma valor, custo e comportamento de compra em uma hipótese comercial testável. Ela prefere o modelo de cobrança que acompanha o valor percebido sem criar medição desnecessária.

Analisa

- comprador e pagador;
- unidade de valor;
- ticket imaginado;
- preço de alternativas;
- benefício financeiro ou operacional;
- custo fixo e variável;
- suporte e esforço de venda;
- implantação, teste e inadimplência;
- MRR, ARPA e clientes para equilíbrio.

Saída

O plano apresenta faixa de preço, premissas, modelo de cobrança e cenários enxuto, base e crescimento. Quando os dados são insuficientes, a especialista marca a faixa como hipótese.

Exemplo

Se o valor nasce da gestão de vários clientes, cobrar por conta da agência pode ser mais simples do que cobrar por cada lead no começo. Essa é uma recomendação a validar, não uma regra universal.

Não faz

Não oferece precisão contábil, não promete margem e não fixa o preço sem considerar custo operacional e disposição de pagamento.

Skill mvpfy - technology

Esta especialista traduz a jornada do MVP em uma arquitetura pequena, operável e coerente com a fase da empresa.

Base recomendada

- Laravel como aplicação principal;
- Node.js somente com justificativa concreta;
- PostgreSQL, com Supabase quando fizer sentido;
- VPS;
- armazenamento compatível com S3;
- e-mail transacional;
- IA externa apenas com função de produto clara;
- backups, logs e tarefas agendadas conforme necessidade.

Entrega

O `MVP.md` recebe arquitetura, dados conceituais, autenticação, papéis, contas, isolamento, integrações, filas, arquivos, segurança básica, observabilidade e custo mensal por cenário.

Exemplo

Para a ferramenta de leads, a arquitetura precisa separar agências e clientes, registrar mudanças de status e enviar notificações. Não precisa começar com microsserviços ou Kubernetes sem um motivo concreto.

Não faz

Não implementa o software, não compra VPS e não trata a stack padrão como requisito que supera o problema do produto.

Skill mvpfy-marketing

Esta especialista prepara a chegada do primeiro SaaS ao mercado. Ela conecta promessa, público, oferta, conteúdo, captação e venda em poucos canais que a empresa consiga manter.

Entrega

- mensagem principal;
- objeções e respostas;
- oferta inicial;
- website mínimo;
- canal prioritário;
- conteúdo por etapa;
- lead magnet quando fizer sentido;
- e-mail marketing;
- grupos e redes sociais;
- demonstração ou venda assistida;
- métricas de aquisição, ativação, conversão e retenção.

Exemplo

Para agências, o primeiro conteúdo pode mostrar como acompanhar um lead desde o contato até o retorno ao cliente. O canal só entra no plano se o público realmente estiver presente e a equipe conseguir manter a rotina.

Não faz

Não recomenda todos os canais, não cria campanha final e não define mensagem sem usar o público e a proposta registrados pelas especialistas anteriores.

Skill `mvpfy-document`

Esta skill renderiza e valida o arquivo final. Ela não decide o conteúdo dos domínios; recebe fatos, escolhas, recomendações, hipóteses e pendências da orquestradora e os organiza conforme o template.

Arquivos usados

- `assets/MVP.template.md` ;
- `scripts/render-company.mjs` ;
- `scripts/validate-company.mjs` ;
- `references/document-rules.md` .

Operação

```
node skills/mvpfy-document/scripts/render-company.mjs --project .  
node skills/mvpfy-document/scripts/validate-company.mjs --project .
```

O renderer localiza IDs estáveis, mantém conteúdo existente e usa "Pendente" quando ainda não houver informação. O validator verifica se as áreas mínimas do primeiro SaaS estão presentes.

Não faz

Não cria `spec.md` , não edita backlog e não apaga uma resposta para substituir por texto genérico.

Skill `mvpfy-migrate`

Esta skill atualiza um `MVP.md` quando o template evolui. Ela usa IDs de seção e versão de schema para inserir o que falta no lugar correto.

Fluxo

1. Ler a versão do documento.
2. Ler IDs existentes.
3. Comparar com o template atual.
4. Inserir seções ausentes.
5. Preservar o conteúdo preenchido.
6. Marcar campos novos como pendentes.
7. Atualizar a versão após gravação válida.
8. Devolver à orquestradora uma única pergunta nova relevante.

Exemplo

Se o template ganhar “Plano de onboarding” e o documento já descreve convite e primeiro valor em outra seção, o texto antigo permanece. A nova seção recebe uma síntese segura ou “Pendente”; o MVPFy pergunta somente o detalhe que não puder ser inferido.

Não faz

Não remove histórico, não duplica seções, não migra arquivos do Specsify e não faz uma rodada de perguntas em lote.

Guia de desenvolvimento do MVPFy



Este percurso explica como o MVPFy funciona por dentro e como alterar a biblioteca sem romper a entrevista, a persistência ou o `MVP.md`. Ele é destinado a agentes e pessoas que contribuem com skills, scripts, template, documentação, ebook ou testes.

O MVPFy é um projeto independente. Ele não importa, cria, lê ou modifica `spec.md`, skills ou arquivos do Specsify. A estrutura desta documentação foi inspirada na separação de percursos do Specsify, mas toda implementação e toda fonte de verdade ficam neste submódulo.

Para aprender a usar o produto em um projeto consumidor, siga o [guia do usuário](#).

Leitura inicial

PRECISO ENTENDER...	DOCUMENTO
arquitetura e responsabilidades	Arquitetura
cada skill e seu handoff	Skills
estado, respostas e gravação	Estado e persistência
template e <code>MVP.md</code>	Template do documento
comandos e arquivos executáveis	Scripts
SemVer, changelog e GitHub Release	Versionamento e releases
testes e validações	Testes
ebook e manutenção das páginas	Documentação e ebook
contribuição completa	Contribuição

Para agentes

1. Leia o `AGENTS.md` da raiz e de `mvpfy/`.
2. Confirme os arquivos executáveis antes de descrever um fluxo.
3. Preserve a separação entre orquestradora e especialistas.
4. Salve a resposta antes de apresentar a pergunta seguinte.
5. Atualize docs, testes e ebook quando a interface pública mudar.
6. Rode os validadores do submódulo antes de entregar a alteração.

Modelo de contribuição

Uma mudança começa pelo comportamento observável. Depois, ajuste a skill ou o script responsável, escreva um teste focal, atualize a documentação de usuário e técnica e reconstrua o ebook. O [mapa de arquivos](#) ajuda a encontrar o owner correto antes da edição.

Contexto técnico

As páginas em [context/](#) registram finalidade, arquitetura, stack, dados, entrevista, documento e testes. Elas são pequenas o pre suficiente para serem carregadas por um agente sem substituir o código ou os contratos canônicos.

Validação rápida

Na raiz do submódulo:

```
npm test
npm run ebook
npm run ebook:verify
```

O lint de Markdown e os validadores da raiz do Hub também fazem parte da entrega quando o submódulo é alterado dentro do monorepo.

Arquitetura do MVPFy

O MVPFy é uma biblioteca de skills instalada em um projeto consumidor. A orquestradora conversa com a pessoa, as especialistas analisam domínios e os scripts cuidam de operações determinísticas.

Fluxo de dados

```
flowchart TD
    A[Pedido do usuário] --> B[mvpfy]
    B --> C[Leitura somente de contexto existente]
    C --> D[Fila de lacunas]
    D --> E[Uma pergunta]
    E --> F[record-answer.mjs]
    F --> G[state.json e answers.jsonl]
    G --> H[Especialista do domínio]
    H --> I[MVP.md]
    I --> J[document e migrate]
```

Responsabilidades

PARTE	RESPONSABILIDADE
mvpfy	Intenção, leitura de contexto, fila, pergunta única e handoff.
Especialistas	Conhecimento e saída estruturada de cada domínio.
mvpfy-document	Renderização e validação do documento.
mvpfy-migrate	Evolução do template sem perda.
Scripts	Persistência, transformação e validação repetíveis.
Testes	Garantia de catálogo, scripts e documento.

Uma pergunta por turno

Esse é um contrato de interface, não uma preferência de redação. A orquestradora pode usar várias fontes internas para decidir a próxima ação, mas a mensagem ao usuário contém somente uma pergunta principal. As opções numeradas respondem a essa mesma pergunta.

Antes de enviar, a skill confere se não está solicitando duas informações, se não repetiu algo presente em uma spec ou backlog e se a gravação do turno anterior ocorreu.

Fontes somente para leitura

No projeto consumidor, a orquestradora pode ler `spec.md`, `specs/`, `backlog/`, `docs/`, `briefs`, `tickets` e planos existentes. Essa leitura reduz a quantidade de perguntas. Nenhum desses arquivos é uma saída do MVPFy e nenhum é criado ou alterado por suas skills.

Fora da arquitetura

O MVPFy não implementa software SaaS, não executa código de produção e não altera o repositório Specsify. O escopo de escrita é `MVP.md`, `.mvpfy/` e, quando a pessoa pedir, os artefatos de documentação do próprio pacote.

Arquitetura das skills

Cada skill vive em seu próprio diretório e mantém uma responsabilidade delimitada. O corpo de `SKILL.md` contém o fluxo essencial; referências guardam regras específicas; scripts cuidam das operações que precisam de repetição exata.

Catálogo completo

SKILL	OWNER	HANDOFF
<code>mvpfy</code>	Conversa, estado e fila	Qualquer especialista ou documento
<code>mvpfy-problem</code>	Situação, problema e hipótese	<code>mvpfy-audience</code>
<code>mvpfy-audience</code>	Segmento e papéis	<code>mvpfy-product</code>
<code>mvpfy-product</code>	Jornada e escopo	<code>mvpfy-saas</code>
<code>mvpfy-saas</code>	Conta e ciclo recorrente	<code>mvpfy-market</code> ou preço
<code>mvpfy-brand</code>	Marca e posicionamento	Marketing ou documento
<code>mvpfy-market</code>	Concorrência e fonte	<code>mvpfy-pricing</code>
<code>mvpfy-pricing</code>	Cobrança e cenários	<code>mvpfy-technology</code>
<code>mvpfy-technology</code>	Stack e operação	<code>mvpfy-marketing</code> ou documento
<code>mvpfy-marketing</code>	Aquisição e venda	Documento
<code>mvpfy-document</code>	MVP.md	Validação
<code>mvpfy-migrate</code>	Evolução de schema	Orquestradora

Estrutura de uma skill

```
skills/<nome>/
├─ SKILL.md
├─ agents/openai.yaml
├─ references/
├─ scripts/
└─ assets/
```

Somente os diretórios necessários devem existir. Não criar README dentro de uma skill. A documentação de uso fica em `docs/user/` e a documentação técnica fica em `docs/develop/`.

Contrato de handoff

Uma especialista recebe fatos conhecidos, escolhas confirmadas, hipóteses, pendências, conflitos e áreas afetadas. Ela devolve fatos novos, escolhas, recomendações, campos resolvidos, lacunas, conflitos e uma possível próxima pergunta.

A especialista não apresenta uma entrevista própria com várias perguntas. A orquestradora escolhe somente um `next_question` para o turno e registra a resposta antes de chamar a próxima etapa.

Alterar uma skill

1. Identifique o comportamento que muda.
2. Atualize `SKILL.md` ou a referência do owner.
3. Adicione ou ajuste teste focal.
4. Atualize páginas de usuário e desenvolvimento.
5. Rode `quick_validate.py` e `npm test`.
6. Recompile o ebook quando a documentação pública mudar.

Não confundir com o Specsify

O padrão de organização desta documentação é uma referência de leitura. As skills do MVPFy não são skills do Specsify, não chamam o método dele e não escrevem `spec.md`.

Contratos

Especialistas recebem um pacote YAML com projeto, pedido, campos conhecidos, escolhas, hipóteses, lacunas, conflitos e modo da entrevista. A resposta retorna fatos, escolhas, recomendações, campos resolvidos, lacunas e `next_question`.

A especialista não deve perguntar várias coisas nem escrever fora da própria área. A orquestradora integra retornos e elimina repetição.

Contrato de uma pergunta

`next_question` representa uma única pergunta principal. A especialista devolve três opções prontas. A orquestradora adiciona:

- 4. Avançar
- 5. Conversar mais sobre este tema

`Avançar` aceita a definição atual. A opção 5 recebe texto livre sobre a mesma pergunta, registra essa conversa e permite reavaliar a lacuna sem apresentar outra pergunta no turno atual.

Estado e persistência

O projeto consumidor guarda a continuidade da entrevista em `.mvpfy/`. A estrutura permite pausar, corrigir uma resposta e continuar sem reiniciar o plano.

Arquivos

ARQUIVO	FUNÇÃO
<code>config.yaml</code>	Projeto, idioma, tipo e template vigente.
<code>state.json</code>	Visão mutável da entrevista e das áreas preenchidas.
<code>answers.jsonl</code>	Eventos append-only com resposta original e interpretação.
<code>research.json</code>	Pesquisas de mercado e fontes consultadas.
<code>template-version</code>	Versão do template usada na última migração.

`state.json`

Os campos centrais são `project_id`, `interview_status`, `active_domain`, `last_question_id`, `answered_question_ids`, `facts`, `choices`, `assumptions`, `recommendations`, `gaps`, `conflicts`, `section_status` e `research_status`.

O estado é uma visão atual. Para saber como uma resposta chegou até ali, leia `answers.jsonl`.

Antes das perguntas fechadas, o evento `intake.initial-idea` guarda a descrição livre do SaaS e do MVP. O estado mantém `interview_stage: initial_idea` até esse evento ser salvo; depois passa para `questions`. Os itens mencionados nessa entrada ficam em `candidate_items` com o status `candidate`.

Evento de resposta

```
{
  "event_id": "uuid",
  "timestamp": "2026-08-16T12:00:00Z",
  "question_id": "problem.context",
  "question_text": "Onde o problema aparece?",
  "raw_answer": "Em clínicas pequenas",
  "normalized_answer": "Clínicas pequenas",
  "extracted_fields": ["problem.context", "audience.segment"],
  "supersedes": null
}
```

Uma correção não apaga o evento anterior. Ela cria outro evento com `supersedes` apontando para o item corrigido.

Gravação segura

O fluxo valida dados, grava um arquivo temporário, substitui o destino e só depois permite a próxima pergunta. Se a gravação falhar, a interface repete o mesmo turno.

Leitura de contexto

Specs, backlogs, docs, briefs e planos existentes podem ser lidos para preencher fatos e evitar repetição. A rotina de leitura deve ser somente de consulta. Ela não deve criar, editar ou migrar arquivos fora de `MVP.md` e `.mvpfy/`.

Template e MVP .md

O template canônico está em skills/mvpfy-document/assets/MVP.template.md . Ele é a fonte estrutural do documento entregue pelo MVPFy.

IDs estáveis

Cada seção começa com um comentário no formato:

```
<!-- mvpfy:section:problem-definition -->
## Problema que o MVP resolve
```

O renderer procura o ID, não o título. Assim, uma mudança de redação não duplica a seção nem apaga o texto preenchido.

Frontmatter

O frontmatter contém `document_type` , `schema_version` , `project_id` , `document_status` , `interview_status` , datas e idioma. O `schema_version` é usado pelo migrador.

Regras de conteúdo

O documento precisa distinguir confirmado, recomendado, hipótese e pendência. Tabelas servem para comparação, listas servem para escopo e parágrafos explicam relações de causa, escolha e consequência.

Renderização e validação

```
node skills/mvpfy-document/scripts/render-company.mjs --project .
node skills/mvpfy-document/scripts/render-company.mjs --project . --check
node skills/mvpfy-document/scripts/validate-company.mjs --project .
```

O primeiro comando cria ou atualiza. `--check` verifica se a renderização precisaria mudar. O validator exige problema, público, proposta, jornada, conta, onboarding, cobrança, escopo, preço, tecnologia, aquisição, métricas e pendências.

Migração

```
node skills/mvpfy-migrate/scripts/migrate-company.mjs --project .
```

O migrador adiciona seções ausentes, preserva o texto e atualiza a versão somente depois de uma gravação válida. Ele nunca toca em `spec.md`, `specs/`, backlogs ou no repositório Specsify.

Scripts e comandos

Os scripts determinísticos ficam dentro da skill que possui o comportamento. Execute-os na raiz do projeto consumidor, salvo indicação diferente.

Preparar o projeto

```
node skills/mvpfy/scripts/setup-project.mjs --project .
```

Cria `.mvpfy/` e `MVP.md` se ainda não existirem. O script é idempotente: arquivos existentes não são substituídos.

Registrar a ideia inicial

```
node skills/mvpfy/scripts/record-initial-idea.mjs \  
  --project . \  
  --idea "Quero um SaaS para acompanhar leads de pequenas agências." \  
  --candidate-items "CRM, aplicativo, agente de IA"
```

Registra a descrição livre antes da primeira pergunta fechada. Os módulos, recursos e integrações citados são guardados como candidatos para investigação, não como requisitos aprovados.

Se a pessoa enviou mais de uma mensagem, execute o comando novamente para acrescentar cada parte. Quando ela escolher a opção 4, conclua a entrada:

```
node skills/mvpfy/scripts/record-initial-idea.mjs --project . --continue
```

Esse comando só libera as perguntas depois que `initial_idea` estiver salvo.

Registrar resposta

```
node skills/mvpfy/scripts/record-answer.mjs \  
  --project . \  
  --question-id problem.context \  
  --question-text "Onde o problema aparece?" \  
  --raw-answer "Em clínicas pequenas" \  
  --normalized-answer "Clínicas pequenas" \  
  --extracted-fields problem.context,audience.segment
```

O comando acrescenta um evento em `answers.jsonl`, atualiza campos do estado e não deve ser chamado depois de uma pergunta que ainda não foi salva.

Renderizar e validar

```
node skills/mvpfy-document/scripts/render-company.mjs --project .  
node skills/mvpfy-document/scripts/validate-company.mjs --project .
```

Migrar

```
node skills/mvpfy-migrate/scripts/migrate-company.mjs --project .
```

Testar o pacote

Na raiz de `mvpfy`:

```
npm test  
npm run ebook  
npm run ebook:verify
```

Os comandos do pacote não editam `specsfy/`. Validações do Hub executadas na raiz são separadas das validações do submódulo.

Mapa de arquivos do MVPFy

Este mapa mostra onde cada responsabilidade vive. Os caminhos são relativos à raiz do submódulo `mvpfy`.

Raiz

ARQUIVO	FINALIDADE
<code>README.md</code>	Visão rápida, instalação, skills e validação.
<code>AGENTS.md</code>	Instruções de contribuição e fontes do submódulo.
<code>LICENSE</code>	Licença do pacote.
<code>package.json</code>	Nome, versão e comandos npm.
<code>docs/</code>	Percursos de usuário, desenvolvimento e especificação.
<code>skills/</code>	Skills distribuídas ao projeto consumidor.
<code>tests/</code>	Testes de catálogo, scripts e documento.
<code>ebooks/</code>	PDF, EPUB e manifesto gerados.
<code>.ebook/</code>	Configuração do pipeline do ebook.
<code>brand/</code>	Ícone e recursos visuais locais do ebook.

Skills

Cada diretório abaixo contém `SKILL.md`, `agents/openai.yaml` e os recursos que seu domínio exige:

DIRETÓRIO	ARQUIVOS RELEVANTES
<code>skills/ mvpfy/</code>	<code>SKILL.md</code> , <code>references/interview-policy.md</code> , <code>references/contracts.md</code> , <code>references/state-schema.md</code> , <code>scripts/setup-project.mjs</code> , <code>scripts/record-initial-idea.mjs</code> e <code>scripts/record-answer.mjs</code> .
<code>skills/ mvpfy- problem/</code>	<code>SKILL.md</code> , <code>references/problem-map.md</code> .
<code>skills/ mvpfy- audience/</code>	<code>SKILL.md</code> , <code>references/audience-map.md</code> .

DIRETÓRIO	ARQUIVOS RELEVANTES
skills/ mvpfy- product/	SKILL.md , references/product-scope.md .
skills/ mvpfy- saas/	SKILL.md , references/saas-lifecycle.md .
skills/ mvpfy- brand/	SKILL.md , references/brand-direction.md .
skills/ mvpfy- market/	SKILL.md , references/market-research.md .
skills/ mvpfy- pricing/	SKILL.md , references/pricing-model.md .
skills/ mvpfy- technology /	SKILL.md , references/technology-baseline.md .
skills/ mvpfy- marketing/	SKILL.md , references/go-to-market.md .
skills/ mvpfy- document/	SKILL.md , assets/MVP.template.md , renderer e validator.
skills/ mvpfy- migrate/	SKILL.md , política e script de migração.

Arquivos comuns de cada skill

ARQUIVO	FUNÇÃO
SKILL.md	Gatilho, instruções essenciais, limites e referências carregáveis.
agents/openai.yaml	Nome exibido, descrição curta e prompt padrão da interface.
references/*.md	Regras detalhadas do domínio, contrato ou política de operação.
scripts/*.mjs	Transformações determinísticas, sem dependência de uma conversa.

ARQUIVO	FUNÇÃO
assets/MVP.template.md	Estrutura canônica copiada e atualizada no projeto consumidor.

Referências por domínio

SKILL	REFERÊNCIA
mvpfy	interview-policy.md , contracts.md , state-schema.md .
mvpfy-problem	problem-map.md .
mvpfy-audience	audience-map.md .
mvpfy-product	product-scope.md .
mvpfy-saas	saas-lifecycle.md .
mvpfy-brand	brand-direction.md .
mvpfy-market	market-research.md .
mvpfy-pricing	pricing-model.md .
mvpfy-technology	technology-baseline.md .
mvpfy-marketing	go-to-market.md .
mvpfy-document	document-rules.md .
mvpfy-migrate	migration-policy.md .

Scripts executáveis

SCRIPT	ENTRADA	SAÍDA
setup-project.mjs	Diretório consumidor.	MVP.md e .mvpfy/ .
record-initial-idea.mjs	Projeto preparado e descrição livre da ideia.	Evento de entrada, ideia inicial e itens candidatos persistidos.
record-answer.mjs	Projeto, pergunta, resposta e campos extraídos.	Evento em answers.jsonl e estado atualizado.
render-company.mjs	Projeto com estado e template.	MVP.md atualizado ou relatório --check .
validate-company.mjs	MVP.md e frontmatter.	Resultado estrutural e estado de prontidão.

SCRIPT	ENTRADA	SAÍDA
<code>migrate-company.mjs</code>	<code>MVP.md</code> e template vigente.	Documento migrado sem duplicação.

Documentação

CAMINHO	PÚBLICO
<code>docs/user/</code>	Pessoa que usa o MVPFy para planejar o SaaS.
<code>docs/user/skills/</code>	Uma página para cada skill disponível.
<code>docs/develop/</code>	Pessoas que alteram o pacote.
<code>docs/develop/context/</code>	Contexto transversal carregável por agentes.
<code>docs/specification.md</code>	Especificação funcional do MVPFy.
<code>docs/reading-order.txt</code>	Ordem única usada pelo ebook.

Pipeline do ebook

ARQUIVO	FINALIDADE
<code>.ebook/build-ebook.sh</code>	Gera PDF, EPUB, aliases e manifesto.
<code>.ebook/pdf.css</code>	Estilo visual do PDF.
<code>.ebook/epub.css</code>	Estilo visual do EPUB.
<code>.ebook/template.html</code>	Template HTML usado pelo PDF.
<code>.ebook/metadata.yaml</code>	Metadados editoriais, idioma e descrição.
<code>.ebook/cover.svg</code>	Arte vetorial da capa.
<code>.ebook/fonts/</code>	Inter, Manrope e licenças das fontes.
<code>brand/logo/icon.svg</code> e <code>icon.png</code>	Ícone usado na capa e na documentação.
<code>ebooks/VERSION</code>	Versão da edição publicada.
<code>ebooks/build.json</code>	Hashes das fontes e dos artefatos gerados.
<code>ebooks/ebook-mvpfy.pdf</code> e <code>.epub</code>	Aliases estáveis da edição vigente.
<code>ebooks/MVPFy-Documentacao-Completa-v*.pdf</code> e <code>.epub</code>	Arquivos versionados de cada edição.

O diretório `.ebook/build/` é temporário e está no `.gitignore`. Os arquivos versionados e os aliases em `ebooks/` são entregas publicadas.

Governança e testes

ARQUIVO	FINALIDADE
<code>AGENTS.md</code>	Regras locais, idioma, fontes e validações.
<code>package.json</code>	Scripts <code>test</code> , <code>setup:example</code> , <code>ebook</code> e <code>ebook:verify</code> .
<code>tests/skills.test.mjs</code>	Catálogo, metadados e contrato de pergunta única.
<code>tests/scripts.test.mjs</code>	Setup, persistência, migração e validação do documento.
<code>tests/release.test.mjs</code>	Sincronização de SemVer e extração das notas.
<code>scripts/validate-release.mjs</code>	Valida <code>VERSION</code> , <code>package.json</code> , changelog e avanço entre commits.
<code>scripts/extract-release-notes.mjs</code>	Extraí a seção da versão para a release do GitHub.
<code>.github/workflows/release.yml</code>	Valida, cria a tag e publica a release após push na <code>main</code> .
<code>CHANGELOG.md</code>	Histórico das versões do pacote e da documentação.
<code>VERSION</code>	Fonte canônica do SemVer do framework.
<code>LICENSE</code>	Licença do projeto.

Arquivos que o consumidor recebe

Ao executar `setup-project.mjs`, o consumidor recebe `MVP.md` e `.mvpfy/`. Ele não recebe `spec.md`, não altera `specsfy/` e não transforma arquivos existentes em artefatos do MVPFy.

Versionamento e releases

O MVPFy usa Semantic Versioning para que cada melhoria publicada tenha uma versão identificável. A fonte canônica é o arquivo `VERSION`. O `package.json` e o `CHANGELOG.md` repetem essa versão para que o pacote, a documentação e o GitHub mostrem o mesmo estado.

Arquivos envolvidos

ARQUIVO	RESPONSABILIDADE
<code>VERSION</code>	Número SemVer que será publicado.
<code>package.json</code>	Versão do pacote Node.js, mantida igual a <code>VERSION</code> .
<code>CHANGELOG.md</code>	Explicação da mudança na seção <code>[X.Y.Z]</code> .
<code>scripts/validate-release.mjs</code>	Confere versão, changelog e avanço desde o commit anterior.
<code>scripts/extract-release-notes.mjs</code>	Prepara as notas usadas no GitHub Release.
<code>.github/workflows/release.yml</code>	Executa testes, cria tag e publica a release.

Escolha da versão

TIPO DE MUDANÇA	EXEMPLO	INCREMENTO
Correção compatível	Ajustar uma validação sem mudar o uso	<code>0.2.0</code> para <code>0.2.1</code>
Nova capacidade compatível	Adicionar uma skill ou comando sem quebrar o fluxo	<code>0.2.0</code> para <code>0.3.0</code>
Quebra de contrato	Remover campo, mudar formato ou exigir migração	<code>0.2.0</code> para <code>1.0.0</code>

O primeiro número zero indica que o framework ainda está antes da primeira versão estável. Mesmo nesse estágio, não reutilize uma versão já publicada.

Como publicar uma melhoria

1. Faça a alteração no framework.
2. Atualize `VERSION` com a próxima versão.
3. Atualize `package.json.version` para o mesmo valor.
4. Crie a seção `## [X.Y.Z] - AAAA-MM-DD` no topo de `CHANGELOG.md`.
5. Explique a alteração e o efeito para quem usa o MVPFy.
6. Rode as verificações locais.
7. Faça commit e push na `main`.

Verifique localmente com:

```
npm run release:check  
npm test
```

Se a mudança alterar `docs/`, também execute:

```
npm run ebook  
npm run ebook:verify
```

O que acontece no GitHub

O workflow roda em todo push na `main`. A publicação só prossegue quando a versão atual é maior que a versão do commit anterior. Depois, ele:

1. executa `npm test`;
2. valida `VERSION`, `package.json` e `CHANGELOG.md`;
3. recusa uma tag `vx.y.z` que já exista;
4. cria e envia a tag anotada;
5. extrai as notas da seção correspondente;
6. cria a GitHub Release com a tag e essas notas.

Isso mantém uma relação direta entre commit, tag, versão e changelog.

Recusas esperadas

O workflow falha quando:

- `VERSION` não usa `MAJOR.MINOR.PATCH` válido;
- `package.json.version` diverge de `VERSION`;
- o changelog não tem a seção da versão atual;
- a versão não avançou em relação ao commit anterior;
- a tag da versão já existe;
- os testes falham.

Corrija a versão no commit seguinte e faça novo push. Não remova tags ou reescreva releases já publicadas para contornar a validação.

Diferença entre framework e ebook

`VERSION` controla o framework. `ebooks/VERSION` controla a edição do manual. Uma alteração de código pode criar uma nova release do framework sem mudar o ebook. Uma alteração documental deve atualizar o ebook conforme as instruções de `AGENTS.md`, além de registrar a versão do framework quando fizer parte da melhoria publicada.

Documentação e ebook

O percurso do usuário, o percurso técnico e a especificação são Markdown. O ebook é um formato derivado dessas páginas, não uma segunda fonte de conteúdo.

Ordem das páginas

`docs/reading-order.txt` lista cada página na ordem pedagógica. O build lê os caminhos, verifica se existem e passa a mesma sequência ao Pandoc para PDF e EPUB.

Build

```
npm run ebook
npm run ebook:verify
```

O primeiro comando gera a edição versionada, os aliases `ebook-mvpfy.pdf` e `ebook-mvpfy.epub` e `ebooks/build.json`. O segundo confere arquivos, hashes, texto mínimo do PDF e integridade do EPUB.

Quando incrementar a versão

Altere `ebooks/VERSION` quando o conjunto de páginas mudar. Uma nova seção ou capítulo usa incremento menor; correção textual sem nova área pode usar patch. O manifesto registra o hash das fontes, portanto o build precisa ser executado depois de toda mudança documental.

Visual

O pipeline usa `pdf.css`, `epub.css`, `template.html`, `metadata.yaml`, capa, fontes e ícone local. O visual segue o padrão dos projetos do Hub sem importar arquivos do Specsfy e sem criar uma fonte paralela no Hub.

Links

Links relativos entre páginas do MVPFy continuam úteis no Markdown e no ebook. Antes do build, confira que toda página referenciada existe.

Testes

Execute:

```
npm test  
npm run ebook:verify
```

Os testes conferem as 12 skills, frontmatter, metadados de interface, IDs do template e os scripts de setup, registro, renderização e migração. Teste manual recomendado: iniciar um projeto temporário, registrar uma resposta, executar a migração duas vezes e conferir que nenhum bloco foi duplicado.

O teste da orquestradora também confirma a regra de uma pergunta por turno, três opções prontas, **Avançar**, conversa livre e leitura somente de fontes existentes do projeto consumidor.

Contribuição

Antes de editar

Leia `AGENTS.md`, confirme o estado Git do submódulo e identifique o owner no [mapa de arquivos](#). Preserve alterações locais que não pertencem à tarefa.

Ao mudar comportamento

1. Descreva o comportamento observável.
2. Altere a skill, referência ou script responsável.
3. Acrescente um teste focal.
4. Atualize a documentação de usuário.
5. Atualize a documentação técnica.
6. Recompile e verifique o ebook se as páginas mudaram.
7. Rode lint, testes e validadores.

Ao mudar uma skill

Use a orientação do `skill-creator`. Confira frontmatter, descrição, estrutura, referências diretas e `agents/openai.yaml`. A regra de uma pergunta por turno precisa aparecer na skill, na política, no contrato e no teste correspondente.

Limites de escrita

Leia specs, backlogs e docs do projeto consumidor quando isso ajudar o MVPFy a evitar perguntas repetidas. Não altere esses arquivos. Dentro do ecossistema, este submódulo só publica a própria biblioteca e seu `MVP.md` no projeto consumidor.

Validação

```
npm test
npm run ebook
npm run ebook:verify
git diff --check
```

Na raiz do Hub, rode também os validadores aplicáveis ao submódulo.

Contexto técnico do MVPFy

Estas páginas registram decisões transversais pequenas para que um agente consiga contribuir sem carregar toda a documentação. Elas explicam o pacote; não são arquivos que o MVPFy instala em consumidores.

TEMA	DOCUMENTO
finalidade e vocabulário	Produto
arquitetura e limites	Arquitetura
entrevista e pergunta única	Entrevista
documento e migração	Documento
stack e operação	Tecnologia
testes e validação	Testes

Contexto do produto

O MVPFy organiza a definição do primeiro SaaS de uma empresa em formação. Seu artefato central é `MVP.md`, não `spec.md`.

O produto recebe ideia, documentos já existentes e respostas do usuário. Ele produz uma síntese com problema, público, escopo, SaaS, preço, tecnologia, marketing e validação.

O MVPFy pode ler specs, backlogs, briefs e planos do projeto consumidor para aproveitar fatos já registrados. Ele nunca cria, modifica ou migra esses arquivos. A referência ao Specsfy vale somente para a organização dos guias e do ebook.

Contexto da entrevista

Cada turno tem uma pergunta principal e cinco opções fixas na saída:

1. resposta pronta;
2. resposta pronta;
3. resposta pronta;
4. Avançar ;
5. Conversar mais sobre este tema .

As opções 1 a 3 são escolhidas pela especialista. A opção 4 aceita o estado atual. A opção 5 abre um chat sobre a mesma pergunta, recebe texto livre ou uma dúvida, salva a resposta e mantém a investigação no mesmo tema. O chat pode ter mais de um turno, mas nunca mostra duas perguntas no mesmo turno.

Contexto do documento

`MVP.md` é gerado pelo template versionado e localizado por IDs estáveis. O renderer preserva seções existentes. O migrador acrescenta seções novas sem substituir decisões anteriores. O validator controla campos mínimos do primeiro SaaS.

Contexto de tecnologia

O padrão de recomendação é Laravel em VPS, com PostgreSQL, Supabase quando adequado, armazenamento S3, e-mail transacional e IA somente quando houver função clara. Node.js é complementar, não padrão obrigatório.

Contexto de testes

Os testes verificam catálogo, metadados de skills, criação de projeto, persistência, migração e validação do `MVP.md`. A documentação também deve ser conferida com Markdown lint e o ebook deve passar pelo manifesto e pelos checks de PDF e EPUB.

Especificação do MVPFy

Finalidade

O MVPFy é uma suíte de skills para uma pessoa que está criando uma empresa e seu primeiro produto SaaS. Ele transforma contexto em um plano de MVP para produto, desenvolvimento, website, marca, marketing, vendas e operação.

O artefato central é `MVP.md`. O MVPFy não cria `spec.md`, não executa o método do Specsify e não altera arquivos do repositório Specsify.

Entrada e fontes

O usuário pode começar com uma frase, uma ideia detalhada, URLs de concorrentes ou arquivos existentes no projeto. A orquestradora lê, em modo somente leitura, `MVP.md`, `.mvpfy/`, `specs`, `backlogs`, `briefs`, `docs`, `tickets` e `planos`. Uma informação clara encontrada nessas fontes conta como contexto preenchido e não deve voltar como pergunta.

Uma mensagem pode conter o plano inteiro conhecido até aquele momento. Antes de cada pergunta, a orquestradora analisa a mensagem completa, os eventos anteriores, o `MVP.md` e as referências disponíveis. Ela registra todos os campos identificados e pergunta somente sobre a lacuna prioritária restante. Mensagens adicionais durante a entrada inicial são acumuladas; a opção 4 continua disponível para iniciar as perguntas fechadas.

Contrato rígido da entrevista

Cada turno tem exatamente uma pergunta principal. A mensagem pode conter uma confirmação curta e contexto, mas termina com exatamente cinco opções:

1. resposta pronta;
2. resposta pronta;
3. resposta pronta;
4. Avançar ;
5. Conversar mais sobre este tema .

As opções 1, 2 e 3 respondem à mesma pergunta. Avançar aceita o entendimento atual e segue para a próxima lacuna. Conversar mais sobre este tema abre texto livre sobre a pergunta atual. A resposta livre é salva, analisada e só então a próxima pergunta pode aparecer em um novo turno.

É inválido exibir duas perguntas, pedir que a pessoa responda dois campos, mostrar um formulário ou adicionar uma sexta opção.

Ciclo de persistência

1. Ler fontes e estado atuais.
2. Escolher uma lacuna prioritária.
3. Exibir a pergunta única e as cinco opções.
4. Receber número ou texto livre.
5. Salvar resposta bruta e interpretação.
6. Atualizar fatos, escolhas, hipóteses e áreas cobertas.
7. Atualizar `MVP.md` quando aplicável.
8. Só então selecionar o próximo turno.

Se a gravação falhar, o MVPFy repete a mesma pergunta. Ele não avança com estado incerto.

Especialistas

SKILL	RESPONSABILIDADE
<code>mvpfy</code>	Orquestração, contexto, estado e pergunta única.
<code>mvpfy-problem</code>	Problema, situação, alternativa e hipótese.
<code>mvpfy-audience</code>	Segmentos, papéis e personas.
<code>mvpfy-product</code>	Proposta, jornada e escopo 1.0.
<code>mvpfy-saas</code>	Conta, acesso, onboarding, cobrança e retenção.
<code>mvpfy-brand</code>	Posicionamento, nome e slogan.
<code>mvpfy-market</code>	Concorrentes, alternativas, fontes e preços observados.
<code>mvpfy-pricing</code>	Modelo de cobrança, faixas e cenários econômicos.
<code>mvpfy-technology</code>	Laravel, VPS, dados, arquivos, IA e operação.
<code>mvpfy-marketing</code>	Website, conteúdo, leads, venda e lançamento.
<code>mvpfy-document</code>	Renderização e validação do <code>MVP.md</code> .
<code>mvpfy-migrate</code>	Atualização do template com preservação.

Escopo do primeiro SaaS

O plano precisa definir um público, um problema, uma promessa e uma jornada principal. Também precisa explicar titular da conta, usuários, primeiro valor, assinatura, suporte, cancelamento, custo, preço e métricas.

Itens interessantes que não sustentam a validação entram fora da versão 1.0. Processos manuais podem aparecer como operação provisória quando reduzem o tempo de construção.

Tecnologia preferencial

Laravel em VPS é a base inicial. PostgreSQL, Supabase, object storage, provedor de e-mail e IA entram conforme a jornada exigir. Node.js só entra com justificativa concreta. A recomendação registra custo, premissa e condição para revisão.

MVP .md

O template possui 35 seções com IDs estáveis. O documento distingue confirmado, recomendado, hipótese e pendência. `preliminary` significa que há lacunas; `ready` significa que os campos mínimos do MVP SaaS estão descritos; `validated` significa que as hipóteses principais foram conferidas fora da entrevista.

Migração

Quando a versão do template muda, `mvpfy-migrate` adiciona seções ausentes, mantém conteúdo, marca campos novos e devolve uma única pendência. Não migra specs, backlogs, `spec.md` ou qualquer arquivo externo ao MVPFy.

Aceite

- uma pergunta principal por turno;
- três opções prontas, `Avançar` e conversa livre;
- gravação antes da pergunta seguinte;
- leitura de contexto existente sem escrita nesses arquivos;
- retomada depois de pausa;
- correção com histórico;
- `MVP.md` idempotente e validável;
- 12 skills com responsabilidades separadas;
- PDF e EPUB reconstruíveis a partir da documentação.